Лабораторная работа №2

Метрические методы классификации

Цель работы

Изучить и реализовать методы машинного обучения для задачи бинарной классификации

Задачи работы

предсказание `isAlive` — жив ли персонаж методами:

- **kNN** — метод (k) ближайших соседей с различными метриками и весами;
- **aNN** — адаптивный метод ближайших соседей (окно Парзена с фиксированной и переменной шириной);
- **ANN** — приближённый поиск ближайших соседей (библиотека `Annoy`) на векторах признаков персонажей;
- методы улучшения: LOO-подбор гиперпараметров, профиль компактности, отбор эталонных объектов, сравнение с логистической регрессией.

Исходные материалы

- Семинар 2: расстояния (Евклид, Минковский), kNN, окно Парзена, метод потенциальных функций, профиль компактности.
- Практика 2: логистическая регрессия, L1/L2-регуляризация, LOO, SGD.
- Вспомогательный ноутбук №1 (`hw_1_game_of_thrones.ipynb`): датасет GoT, предобработка, `isAlive` как целевая переменная.
- Вспомогательный ноутбук №2 `addition_KNN_ANN.ipynb`: реализация kNN, `AnnoyIndex`, `LabelEncoder`.

Датасет

- `game_of_thrones_train.csv` — обучающая выборка (1557 строк, 26 столбцов);
- `game_of_thrones_test.csv` — тестовая выборка (389 строк, 25 столбцов, без `isAlive`).

Ссылки:

- `https://raw.githubusercontent.com/EvADarya/Data_processing-DLS-/refs/heads/main/game_of_thrones_train.csv`
- `https://raw.githubusercontent.com/EvADarya/Data_processing-DLS-/refs/heads/main/game_of_thrones_test.csv`

Задания

Часть 1. Подготовка данных (общая для всех вариантов, но с вариациями)

1. Загрузите обучающий и тестовый датасеты GoT.
2. Проведите предобработку согласно **вашему варианту** (см. таблицу ниже):
   - обработка пропусков (`age` , `dateOfBirth` , `culture` , `house` и т.д.);
   - **способ кодирования** `popularity` (`log` / `qcut` / `raw` — см. вариант);
   - **способ группировки** `culture` (полная группировка из ДЗ / без группировки / объединение редких категорий);
   - **кодирование** `house` (`LabelEncoder` / `OneHotEncoder` / top-K домов + `other`);
   - масштабирование числовых признаков (`StandardScaler`).
3. Разделите обучающую выборку на train/val/test (70/15/15) с `random_state = seed` (см. вариант).

Часть 2. kNN классификация

1. Реализуйте **собственную** функцию `knn_classify` (без `sklearn`) с поддержкой:
   - метрики: **см. вариант** (`euclidean` / `manhattan` / `minkowski` с (p=3));
   - весов: **см. вариант** (`uniform` / `distance` / `linear` / `exponential`).
2. Исследуйте зависимость accuracy от (k) в диапазоне (1\ldots k_{max}) (см. вариант).
3. Выберите оптимальное (k) с помощью **leave-one-out (LOO)** на тренировочной выборке.
4. Оцените accuracy, f1_score на валидационной выборке для лучшего (k) и для двух других типов весов.
5. Сравните с `KNeighborsClassifier` из `sklearn` при том же (k).

Часть 3. aNN (окно Парзена)

1. Реализуйте `parzen_classify` с ядром **см. вариант** (`gaussian` / `rectangular` / `triangular` / `epanechnikov` / `quartic`).
2. Реализуйте два режима:
   - фиксированная ширина окна (h) (диапазон (h): от 0.05 до (h_{max}), см. вариант);
   - переменная ширина (aNN) — ширина определяется расстоянием до (k)-го соседа.
3. Подберите оптимальные (h) или (k) по LOO.
4. Визуализируйте решающие границы на синтетических данных (`make_moons`) для 3 значений (h).
5. Оцените accuracy, f1_score на валидационной выборке GoT.

Часть 4. Профиль компактности и отбор эталонных объектов

1. Постройте профиль компактности (\Pi(m)) для тренировочной выборки GoT (с использованием метрики из варианта).
2. Реализуйте жадное удаление эталонных объектов для 1NN (с критерием LOO).
3. Оцените, как меняется accuracy и размер выборки Ω.
4. Визуализируйте результат (например, t-SNE или первые 2 признака после PCA).

Часть 5. Метод потенциальных функций

1. Реализуйте `potential_classify` с ядром **см. вариант** и фиксированным (h).
2. Сравните с kNN и aNN по accuracy, f1_score.
3. Оцените влияние (h) на качество.

Часть 6. ANN (приближённый поиск ближайших соседей)

1. Установите библиотеку `annoy`.
2. Возьмите **векторы признаков** персонажей GoT (после предобработки) и постройте `AnnoyIndex` с метрикой из варианта и `n_trees` (см. вариант).
3. Для **заданного персонажа** (см. `query_character` в варианте) найдите `n_neighbors_ann` ближайших соседей из обучающей выборки.
4. Сравните результат с точным поиском (`sklearn.neighbors.NearestNeighbors` или brute-force перебор по всей выборке).
5. Оцените:
   - долю совпавших соседей (recall@k);
   - время поиска ANN vs точного поиска.
6. Исследуйте, как `n_trees` влияет на скорость, точность, f1_score.

Часть 7. Сравнение с линейными моделями

1. Обучите `LogisticRegression` с L2-регуляризацией; подберите (\gamma) по LOO (как в практике №2).
2. Сравните accuracy с лучшими непараметрическими моделями.
3. Сделайте вывод, в каких случаях непараметрические методы выигрывают, а в каких — проигрывают.

Часть 8. Формирование ответа

1. Примените лучшую модель к тестовому датасету `game_of_thrones_test.csv`.
2. Сформируйте `submission.csv` с предсказаниями `isAlive`.

Таблица вариантов 1–20

Вар.	seed	k_{max}	h_{max}	Метрика kNN	Веса kNN	Ядро Parzen	Popularity	Culture	House	n_trees	(n_neighbors_ann)	query_character
1	1	15	1.0	euclidean	uniform	gaussian	log(M=100)	полная	LabelEncoder	5	5	Jon Snow
2	2	16	1.1	manhattan	distance	rectangular	qcut(5)	полная	OneHot	6	6	Daenerys Targaryen
3	3	17	1.2	minkowski(p=3)	linear	triangular	log(M=50)	без группировки	top-10 + other	7	7	Tyrion Lannister
4	4	18	1.3	euclidean	exponential	epanechnikov	log(M=100)	полная	LabelEncoder	8	8	Arya Stark
5	5	19	1.4	manhattan	uniform	quartic	qcut(10)	объединение редких	OneHot	9	9	Sansa Stark
6	6	20	1.5	minkowski(p=3)	distance	gaussian	log(M=200)	полная	top-10 + other	10	10	Bran Stark
7	7	14	1.0	euclidean	linear	rectangular	log(M=100)	без группировки	LabelEncoder	11	5	Cersei Lannister

Вар.	seed	(k_{max})	(h_{max})	Метрика kNN	Веса kNN	Ядро Parzen	Popularity	Culture	House	n_trees	(n_neighbors_ann)	`query_character`
8	8	13	1.1	manhattan	exponential	triangular	qcut(5)	полная	OneHot	12	6	Jaime Lannister
9	9	12	1.2	minkowski(p=3)	uniform	epanechnikov	log(M=50)	объединение редких	top-10 + other	13	7	Ned Stark
10	10	11	1.3	euclidean	distance	quartic	log(M=100)	полная	LabelEncoder	14	8	Rob Stark
11	11	15	1.4	manhattan	linear	gaussian	qcut(10)	без группировки	OneHot	15	9	Theon Greyjoy
12	12	16	1.5	minkowski(p=3)	exponential	rectangular	log(M=200)	полная	top-10 + other	6	10	Stannis Baratheon
13	13	17	1.0	euclidean	uniform	triangular	log(M=100)	объединение редких	LabelEncoder	7	5	Davos Seaworth
14	14	18	1.1	manhattan	distance	epanechnikov	qcut(5)	полная	OneHot	8	6	Samwell Tarly
15	15	19	1.2	minkowski(p=3)	linear	quartic	log(M=50)	без группировки	top-10 + other	9	7	Brienne of Tarth

Вар.	seed	(k_{max})	(h_{max})	Метрика kNN	Веса kNN	Ядро Parzen	Popularity	Culture	House	n_trees	(n_neighbors_ann)	`query_character`
16	16	20	1.3	euclidean	exponential	gaussian	log(M=100)	полная	LabelEncoder	10	8	Sandor Clegane
17	17	14	1.4	manhattan	uniform	rectangular	qcut(10)	объединение редких	OneHot	11	9	Petyr Baelish
18	18	13	1.5	minkowski(p=3)	distance	triangular	log(M=200)	полная	top-10 + other	12	10	Varys
19	19	12	1.0	euclidean	linear	epanechnikov	log(M=100)	без группировки	LabelEncoder	13	5	Jorah Mormont
20	20	11	1.1	manhattan	exponential	quartic	qcut(5)	полная	OneHot	14	6	Melisandre

---

Пояснение к столбцам таблицы

`seed`

Число, используемое для инициализации генераторов случайных чисел (`random_state` в `train_test_split`, `numpy.random.seed`, инициализация `AnnoyIndex` при необходимости). Гарантирует воспроизводимость результатов: при одном и том же `seed` разбиение train/val и все случайные операции дают одинаковый результат у всех студентов с одним вариантом.

(k_{max})

Максимальное число соседей (k), до которого нужно перебирать гиперпараметр в kNN. Перебор идёт по (k \in {1, 2, \ldots, k_{max}}). Оптимальное (k) выбирается по LOO на тренировочной выборке — это то

значение, при котором LOO-ошибка минимальна. Построить графики зависимости accuracy и f1_score от (k) (как в семинаре).

(h_{max})

Верхняя граница диапазона ширины окна (h) для метода Парзена (aNN). Перебор (h) идёт по `np.linspace(0.05, h_max, N)`. Для каждого (h) считается LOO-ошибка; оптимальное (h) — то, которое даёт минимум. Это аналог (k_{max}), но для ядерного классификатора.

Метрика kNN

Метрика расстояния между объектами в kNN и в профиле компактности:

- `euclidean` — (d(x, y) = \sqrt{\sum_i (x_i - y_i)^2}), стандартный выбор;
- `manhattan` — (d(x, y) = \sum_i |x_i - y_i|), устойчивее к выбросам;
- `minkowski(p=3)` — (d(x, y) = \left(\sum_i |x_i - y_i|^{p\right)}{1/p}) с (p=3).

При реализации собственного `knn_classify` нужно реализовать обобщённую метрику Минковского с параметром (p), чтобы одна и та же функция работала для всех вариантов.

Веса kNN

Способ учёта вклада соседей при голосовании:

- `uniform` — все соседи голосуют одинаково ((w_i = 1));
- `distance` — вес обратно пропорционален расстоянию ((w_i = 1 / d_i));
- `linear` — линейно убывает с номером соседа ((w_i = (k + 1 - i) / k));
- `exponential` — экспоненциально убывает ((w_i = q^{i-1}), (q = 0.5)).

Все четыре типа нужно реализовать в одной функции и сравнить их на валидационной выборке при лучшем (k).

Ядро Parzen

Функция (K®), определяющая вклад соседа в зависимости от нормированного расстояния (r = d / h):

- `gaussian` — (K® = \exp(-2 r^2)), гладкое ядро без финитного носителя;
- `rectangular` — (K® = \mathbb{1}[|r| \le 1]), окно фиксированного радиуса;
- `triangular` — (K® = (1 - |r|) \cdot \mathbb{1}[|r| \le 1]), линейно убывает;
- `epanechnikov` — (K® = (1 - r^2) \cdot \mathbb{1}[|r| \le 1]), оптимальное по СКО;
- `quartic` — (K® = (1 - r^{2)}2 \cdot \mathbb{1}[|r| \le 1]), биквадратное ядро.

Каждый вариант получает своё ядро — это позволяет сравнить различные подходы к взвешиванию.

Popularity

Способ обработки признака `popularity` (число внутренних ссылок в вики), распределение которого сильно несимметрично:

- `log(M=k)` — логарифмическое преобразование `np.log10(popularity * k + 1)`. Прибавление 1 нужно, так как у части персонажей `popularity == 0`. Параметр (M) (100, 50, 200) масштабирует признак перед логарифмом.
- `qcut(n)` — дискретизация на (n) квантильных бинов (`pd.qcut`), превращающая числовой признак в категориальный. Требует последующего One-Hot кодирования.
- `raw` — без преобразования (в таблице не встречается, но допустимо).

Culture

Способ обработки признака `culture` (принадлежность к народу):

- **полная** — использовать группировку из ДЗ (`cultures_grouped`): Old Nations, the North, the Iron Islands, the Mountain and the Vale, the Isles and Rivers, the Rock, the Stormlands, the Reach, Dorne, Essos Nations, Other Nations + `culture_no_data`.
- **без группировки** — использовать `culture` как есть, закодировав через `LabelEncoder` (много редких категорий, модель может переобучиться).
- **объединение редких** — оставить top-K популярных народов, остальные заменить на `other`.

House

Способ кодирования признака `house` (дом персонажа), в котором сотни уникальных значений:

- `LabelEncoder` — каждому дому присваивается целое число (недостаток: модель может решить, что «Дом 5» больше «Дома 2» — это ложная ordinal-зависимость).
- `OneHot` — каждому дому соответствует отдельный бинарный признак (размерность сильно растёт, но нет ложного порядка).
- `top-10 + other` — 10 самых частых домов кодируются как отдельные категории, все остальные объединяются в `other` (компромисс между размерностью и информативностью).

`n_trees`

Количество деревьев в `AnnoyIndex`. Это гиперпараметр ANN-индекса:

- больше деревьев → точнее поиск (recall@k выше), но больше время построения индекса и потребление памяти;
- меньше деревьев → быстрее, но менее точный поиск.

Исследуйте зависимость recall@k и времени поиска от `n_trees`, построив график.

`n_neighbors_ann`

Число ближайших соседей, которое нужно извлечь из `AnnoyIndex` для заданного персонажа. Используется для оценки recall@k относительно точного поиска (`sklearn.neighbors.NearestNeighbors` с тем же (k)): `recall@k = |ANN_top_k ∩ exact_top_k| / k`.

`query_character`

Имя персонажа из **обучающей** выборки GoT, для которого ищутся ближайшие соседи. Персонаж должен присутствовать в `game_of_thrones_train.csv` в столбце `name`. Если точного совпадения нет (например, у вас другой датасет или это тестовый персонаж), выберите ближайшего по алфавиту и укажите это в отчёте.

Алгоритм работы:

1. Возьмите вектор признаков персонажа `query_character` после предобработки.
2. Постройте `AnnoyIndex` на векторах **всех** персонажей обучающей выборки (кроме самого query, если он там есть).
3. Извлеките `n_neighbors_ann` ближайших соседей.
4. Сравните их с результатом точного поиска через `sklearn.neighbors.NearestNeighbors`.

Требования к отчёту

Формат сдачи

1. **Вся работа выполняется в отдельных файлах `*.py`** (не в Jupyter Notebook).
   - Рекомендуется разбить работу на модули по функционалу.
   - Допускается единый `main.py` со всеми частями и функцией `main()`, запускающей весь пайплайн.
2. **Все исходные файлы работы (`*.py` , `submission.csv` , `README.md`) загружаются на GitHub.**
   - Репозиторий должен быть **публичным** (или добавьте преподавателя в collaborators).
   - В `README.md` репозитория укажите: ФИО, номер группы, номер варианта, краткое описание структуры проекта, инструкцию по запуску.
   - Не забудьте загрузить вспомогательные файлы, если они нужны (например, сохранённые `.pkl`-модели или результаты).
   - Ссылка на репозиторий отправляется вместе с отчётом.

Оформление отчёта

3. **Отчёт оформляется в формате `*.docx`** (Microsoft Word или Google Docs → экспорт в `.docx`).
   - Титульный лист: название вуза, дисциплина, номер лабораторной работы, ФИО, группа, номер варианта, дата.
   - Оглавление.
   - Описание используемого датасета и варианта (все гиперпараметры из таблицы).

- По каждой части: постановка задачи, ключевые фрагменты кода (скриншотами или вставленным кодом), графики, таблицы результатов, краткий вывод.
- В конце — общий **вывод по всей работе**.
- Ссылка на GitHub-репозиторий с кодом.
- Отчёт сдаётся в виде одного файла `MethAlg_ML_Lab2_report_group.docx`.

Требования к содержанию

4. **Код должен быть воспроизводимым** (фиксированные `seed` / `random_state`).
5. **Все ключевые шаги сопровождаются комментариями** (без фанатизма — не нужно комментировать каждую строку).
6. **Графики должны иметь заголовки, подписи осей и легенды.**
7. **В конце каждого раздела — краткий вывод.**
8. **Итоговый вывод (обязателен)** в конце отчёта и в виде многострочного комментария в конце основного `*.py` файла. В выводе должно быть отражено:
   - сравнение методов (kNN, aNN, потенциальные функции, логистическая регрессия) по accuracy, f1_score на валидационной выборке;
   - рекомендации по выбору гиперпараметров ((k), (h), веса, метрика, ядро);
   - влияние `n_trees` на точность, f1_score и скорость поиска в ANN;
   - какие признаки и способы их кодирования дали наибольший вклад в качество;
   - в каких случаях непараметрические методы выигрывают у линейных, а в каких — проигрывают;
   - какие сложности возникли и как они были решены.

Критерии оценивания

Баллы	Критерий
0–28	Работа не сдана или выполнена менее чем на 50%
29–34	Выполнены части 1–3, но есть ошибки в реализации kNN/aNN
35–39	Выполнены все части, но подбор гиперпараметров или LOO сделан с недочётами
40–44	Все задания выполнены, графики корректны, сделаны содержательные выводы; код оформлен в `*.py` и загружен на GitHub
45–50	Работа выполнена на отлично, проявлена самостоятельность, код оптимизирован, проведено сравнение с линейными моделями и ANN; репозиторий структурирован (модули, `README.md`), выводы подробные и обоснованные при защите и в отчете

Удачи в выполнении работы!